

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4

Выполнил:

Машковцева Марина

K3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить существующее монолитное бэкенд-приложение (сервис обмена рецептами) на микросервисы, придерживаясь подхода database-per-service (каждый микросервис владеет своей базой данных). Необходимо спроектировать архитектуру, определить способы взаимодействия сервисов, описать новые эндпоинты для межсервисного взаимодействия в формате OpenAPI, а также задокументировать возможные ошибки. Результат должен содержать конкретные шаги для декомпозиции монолита.

Ход работы

1. Выделение микросервисов

Микросервис	Ответственность	Таблицы (БД)
<i>User Service</i>	Управление пользователями, аутентификация, JWT-токены, профили	user
<i>Recipe Service</i>	Рецепты, шаги, медиа, категории, сложности	recipe, step
<i>Social Service</i>	Комментарии, лайки, сохранения, подписки	comment, like, saved_recipe, subscription

Пояснение:

- User Service - независимый домен аутентификации и профилей.
- Recipe Service - основная предметная область (рецепты). Не зависит от социальных действий.
- Social Service - все взаимодействия пользователей с рецептами вынесены отдельно, чтобы можно было масштабировать независимо.

2. Взаимодействие микросервисов

Вызывающий сервис	Целевой сервис	HTTP метод	Внутренний эндпоинт	Когда вызывается
<i>Recipe Service</i>	User Service	GET	/internal/users/{id}	При отображении рецепта нужно получить имя и аватар автора
<i>Recipe Service</i>	Social Service	GET	/internal/recipes/{id}/stats	При формировании списка рецептов или детальной страницы для получения количества лайков и комментариев
<i>Social Service</i>	User Service	GET	/internal/users/{id}	При добавлении комментария/лайка/подписки – проверить, что пользователь существует
<i>Social Service</i>	Recipe Service	GET	/internal/recipes/{id}	При сохранении рецепта, лайке или комментарии – проверить, что рецепт существует
<i>API Gateway</i>	любой сервис	согласно внешним маршрутам	внешние эндпоинты (/api/recipes, /api/auth/login и т.д.)	Все запросы от клиента

3. Разделение баз данных

Каждый сервис использует свою базу данных. Таблицы не пересекаются.

- User DB: user
- Recipe DB: recipe, step
- Social DB: comment, like, saved_recipe, subscription

Миграция данных: при разделении монолита существующие данные необходимо распределить по новым БД. Для этого нужно:

- Создать дампы соответствующих таблиц.
- Импортировать в целевые БД.
- Удалить лишние таблицы из каждой БД.

4. Эндпоинты для межсервисного взаимодействия (OpenAPI)

Сервисы будут вызывать друг друга по внутренним эндпоинтам (недоступным извне). Опишем их в формате OpenAPI.

4.1 Эндпоинты, которые предоставляет User Service для других сервисов

openapi: 3.0.0

info:

title: User Service Internal API

version: 1.0.0

paths:

/internal/users/{id}:

get:

summary: Получить информацию о пользователе по ID

parameters:

- name: id

in: path

required: true

schema:

type: integer

responses:

'200':

description: Успешно

content:

application/json:

schema:

type: object

properties:

id:

type: integer

username:

type: string

email:

type: string

avatar_url:

type: string

'404':

description: Пользователь не найден

4. 2 Эндпоинты, которые предоставляет Recipe Service для других сервисов

paths:

/internal/recipes/{id}:

get:

summary: Проверить существование рецепта и получить базовую информацию

parameters:

- name: id

in: path

required: true

schema:

type: integer

responses:

'200':

description: Рецепт существует

content:

application/json:

schema:

type: object

properties:

id:

type: integer

title:

```
    type: string
  author_id:
    type: integer
'404':
  description: Рецепт не найден
```

4.3 Эндпоинты, которые предоставляет Social Service для других сервисов

paths:

```
/internal/recipes/{id}/stats:
```

get:

summary: Получить количество лайков и комментариев для рецепта

parameters:

- name: id

in: path

required: true

schema:

type: integer

responses:

'200':

description: Статистика

content:

application/json:

schema:

type: object

properties:

likes_count:

type: integer

comments_count:

type: integer

5. Варианты запросов и ответов, возможные ошибки

Пример запроса от Recipe Service к Social Service:

GET /internal/recipes/123/stats

Ответ (200):

```
{  
  "likes_count": 15,  
  "comments_count": 4  
}
```

Возможные ошибки:

- 404 Not Found - рецепт не найден в социальном контексте (нет лайков/комментов).
- 500 Internal Server Error - внутренняя ошибка сервиса.

Пример проверки существования пользователя (User Service):

GET /internal/users/42

Ответ (200):

```
{  
  "id": 42,
```



```
"username": "chef",  
  
"email": "chef@example.com",  
  
"avatar_url": "/avatars/42.jpg"  
  
}
```

Возможная ошибка:

Ошибка 404 - пользователь не существует.

6. Шаги для достижения разделения монолита

1. Создать три отдельных проекта (папки) на основе существующего кода, используя тот же стек.
2. Настроить разные базы данных для каждого сервиса.
3. Переписать контроллеры, удалив прямые SQL-запросы к чужим таблицам. Вместо этого добавить HTTP-клиент для вызова внутренних эндпоинтов соседних сервисов.
4. Настроить API Gateway, который будет проксировать запросы:
 - /api/auth/* => User Service
 - /api/recipes/* => Recipe Service
 - /api/comments/*, /api/likes/*, /api/users/*/subscribe => Social Service
5. Обновить переменные окружения (URL соседних сервисов, порты).
6. Протестировать взаимодействие в среде разработки.

Вывод

В результате работы спроектирована микросервисная архитектура для сервиса обмена рецептами. Выделены три сервиса (User, Recipe, Social), каждый со своей базой данных. Описаны внутренние эндпоинты для синхронного взаимодействия в формате OpenAPI. Предложены конкретные шаги по декомпозиции существующего монолита.